

SIC AI Chapter Quiz: 08

Student Name: Devesh Panwar

Candidate ID: SIC202500060

Question 01

- **Answer:** Option 2 ("Complex decision-making boundaries cannot be created...")
- **Reasoning:**
 - Neural networks need a sufficient number of hidden nodes to capture non-linear relationships and complex patterns in data.
 - If the node count is too low, the model becomes too simple (low capacity) and suffers from **Underfitting**, meaning it can only draw simple lines or curves rather than the complex shapes needed for real-world data.

Question 02

- **Answer:** Tensor
- **Explanation:**
 - A **Tensor** is the standard data structure used in deep learning frameworks (like TensorFlow) to represent all data (inputs, weights, outputs).
 - It is a generalization of vectors and matrices to higher dimensions.
 - **Rank:** Number of dimensions.
 - **Shape:** Size of each dimension.
 - **Type:** Data type of the elements (e.g., float, int).

Question 03

- **The Phenomenon: Overfitting.**
 - This occurs when the model fits the training data too closely, learning noise instead of the underlying pattern. As a result, performance on the verification (test) data degrades (error rises) even as training error decreases.
- **The Solution:**
 1. **Early Stopping:** Stop training at the epoch where the verification error is lowest, before it starts to rise.
 2. **Regularization:** Use techniques like **Dropout** or **L2 Regularization** (Weight Decay).
 3. **More Data:** Increase the amount of training data to improve generalization.

Question 04

```
import torch
import torch.nn as nn

# --- Case (a): 2D Convolution ---
# Input: Batch=1, Channels=3 (k), Height=10, Width=10
# Kernel: 3x3. Note that PyTorch automatically sets kernel depth to match input channels (3).
conv2d = nn.Conv2d(in_channels=3, out_channels=1, kernel_size=3)
input_2d = torch.randn(1, 3, 10, 10)
output_2d = conv2d(input_2d)
print(f"Case (a) Output Shape: {output_2d.shape}")
# Result: [1, 1, 8, 8] -> It is a 2D map (Height x Width)

# --- Case (b): 3D Convolution ---
# Input: Batch=1, Channels=1, Depth=10 (s), Height=10, Width=10
# Kernel: Depth=3 (k), Height=3, Width=3
conv3d = nn.Conv3d(in_channels=1, out_channels=1, kernel_size=(3, 3, 3))
input_3d = torch.randn(1, 1, 10, 10, 10)
output_3d = conv3d(input_3d)
print(f"Case (b) Output Shape: {output_3d.shape}")
# Result: [1, 1, 8, 8, 8] -> It is a 3D Volume (Depth x Height x Width)
```

Case (a) Output Shape: torch.Size([1, 1, 8, 8])
Case (b) Output Shape: torch.Size([1, 1, 8, 8, 8])

Question 05

Final Answer:

$(0.1325, 0.6563, 0.0889, 0.1223)^T$

```
import numpy as np

def softmax(x):
    e_x = np.exp(x - np.max(x)) # Subtract max for numerical stability
    return e_x / e_x.sum(axis=0)

# Input Vector
x = np.array([0.4, 2.0, 0.001, 0.32])

# Apply Softmax
result = softmax(x)

print("--- Chapter 8, Question 5 ---")
print(f"Input Vector: {x}")
print(f"Softmax Result: {result}")
print("-" * 30)
# Formatted for easy reading
print(f"1. {result[0]:.4f}")
print(f"2. {result[1]:.4f}")
print(f"3. {result[2]:.4f}")
print(f"4. {result[3]:.4f}")

--- Chapter 8, Question 5 ---
Input Vector: [4.0e-01 2.0e+00 1.0e-03 3.2e-01]
Softmax Result: [0.13250053 0.65627943 0.08890663 0.12231341]
-----
1. 0.1325
2. 0.6563
3. 0.0889
4. 0.1223
```

Question 06

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
import matplotlib.pyplot as plt

# --- 1. Data Loading (Simulated for this solution) ---
# Since we don't have 'data_boston.csv', we create a dummy sine wave timeseries
# In your real code, keep the line: df = pd.read_csv('data_boston.csv', ...)
data = np.sin(np.linspace(0, 100, 1000)) + np.random.normal(0, 0.1, 1000)
df = pd.DataFrame(data, columns=['PRICE'])

# --- 2. Preprocessing (From your image) ---
scaler = MinMaxScaler()
ts_scaled = scaler.fit_transform(df[['PRICE']])

# Reshape to (Batch_Size, Timesteps, Features) -> (1, 1000, 1)
ts_scaled_2 = ts_scaled.reshape(1, -1, 1)

# --- 3. Prepare X and Y for Training ---
# We want to predict the NEXT step.
# X (Input): From index 0 to end-1
# Y (Target): From index 1 to end
X_train = ts_scaled_2[:, :-1, :]
y_train = ts_scaled_2[:, 1:, :]

# --- 4. Training Parameters (From your image) ---
batch_size = 1
n_epochs = 10 # Reduced to 10 for quick testing (Image says 1000)
learn_rate = 0.0001

# --- 5. Build the Model (COMPLETING THE CODE) ---
model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(None, 1)))

# [Add RNN Layer]
```

```
# return_sequences=True is critical here because we want an output for every timestep
model.add(tf.keras.layers.SimpleRNN(units=50, activation='tanh', return_sequences=True))

# [Add Output Layer]
model.add(tf.keras.layers.Dense(units=1))

# [Compile]
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learn_rate),
              loss='mean_squared_error')

model.summary()

# --- 6. Train the Model ---
print("\nStarting Training...")
history = model.fit(X_train, y_train, epochs=n_epochs, batch_size=batch_size, verbose=1)

# --- 7. Visualization (Optional but recommended) ---
plt.plot(history.history['loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, None, 50)	2,600
dense (Dense)	(None, None, 1)	51

Total params: 2,651 (10.36 KB)
Trainable params: 2,651 (10.36 KB)
Non-trainable params: 0 (0.00 B)

```
Starting Training...
Epoch 1/10
1/1 ----- 1s 1s/step - loss: 0.0086
Epoch 2/10
1/1 ----- 0s 118ms/step - loss: 0.0073
Epoch 3/10
1/1 ----- 0s 116ms/step - loss: 0.0063
Epoch 4/10
1/1 ----- 0s 143ms/step - loss: 0.0057
Epoch 5/10
1/1 ----- 0s 153ms/step - loss: 0.0053
Epoch 6/10
1/1 ----- 0s 118ms/step - loss: 0.0051
Epoch 7/10
1/1 ----- 0s 117ms/step - loss: 0.0050
Epoch 8/10
1/1 ----- 0s 117ms/step - loss: 0.0050
Epoch 9/10
1/1 ----- 0s 117ms/step - loss: 0.0049
Epoch 10/10
1/1 ----- 0s 119ms/step - loss: 0.0049
```

